

Canonical Labelling and Graph Isomorphism: A C++ Reimplementation of Nauty

Auteur: Rodolphe Prévot

Promoteur: Robin Petit

Directeur: Gwenaël Joret

Université Libre de Bruxelles
Département d'Informatique

Septembre 2026



① Contexte

- Définition
- Propriétés intéressantes

② Nauty

- Construction de l'arbre
- Stratégies d'élagages

③ Experimentation

④ Conclusion

Contexte



- Problème assez connu en théorie des graphes.
- Principe: Vérifier si deux graphes sont isomorphes.
- Plusieurs approches pour résoudre: forme canonique la plus efficace en pratique. Remplace un problème d'équivalence en un problème d'égalité.

Définition

Isomorphisme (informelle)

Un *isomorphisme* est une bijection entre deux structures qui préserve les relations entre leurs éléments.

Automorphisme

Un *automorphisme* est un isomorphisme d'un graphe sur lui-même.

Forme canonique

La *forme canonique* $C(G, \pi)$ d'un graphe coloré est un représentant unique de la classe d'isomorphisme de (G, π) . Deux graphes colorés sont isomorphes si et seulement s'ils ont la même forme canonique.

Exemple isomorphisme (1/2)

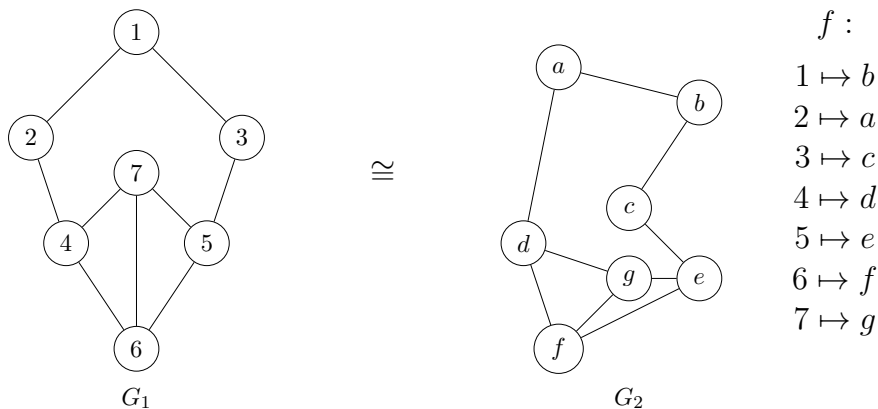
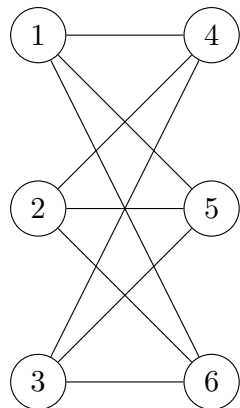


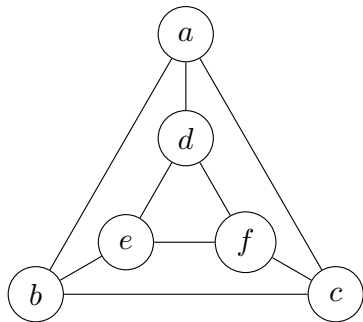
Figure 1: Isomorphisme possible

Exemple isomorphisme (2/2)



G_1

$\not\cong$



G_2

Figure 2: Isomorphisme impossible

Propriétés intéressantes

Complexité

GI appartient à la classe NP, mais on ne sait pas encore s'il est NP-complet ou s'il appartient à P. Il est largement admis que GI est un problème NP-intermédiaire, c'est-à-dire qu'il n'est ni NP-complet ni dans P.

Plus rapide algorithme connu (théorique)

GI peut être résolu en temps quasi-polynomial: $e^{\log(n)^{O(1)}}$ Ce résultat renforce l'idée que GI pourrait être un problème NP-intermédiaire.

En pratique

Ce sont plutôt les algorithmes de *individualisation-raffinement* qui sont utilisés (*Nauty*). Tous les algorithmes appartenant à cette famille ont une borne inférieure exponentielle. Théorie $\neq$ Pratique.

Nauty

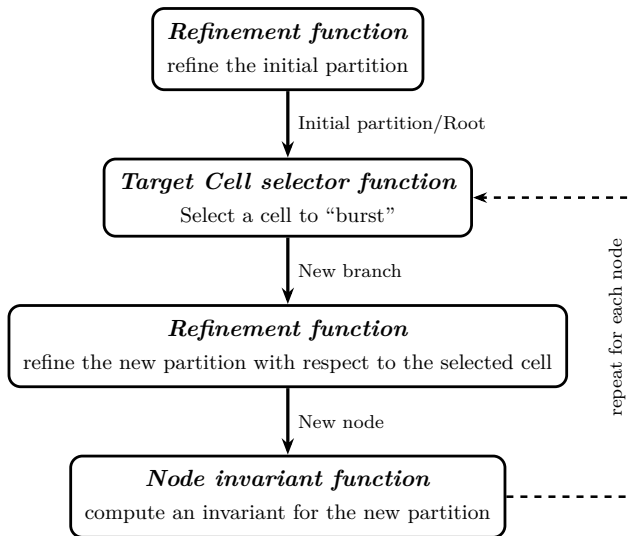


- *Nauty* est basé sur le "colour refinement".
- Énormément de fonctionnalités.
- Calcul de la forme canonique basé sur un arbre de recherche.

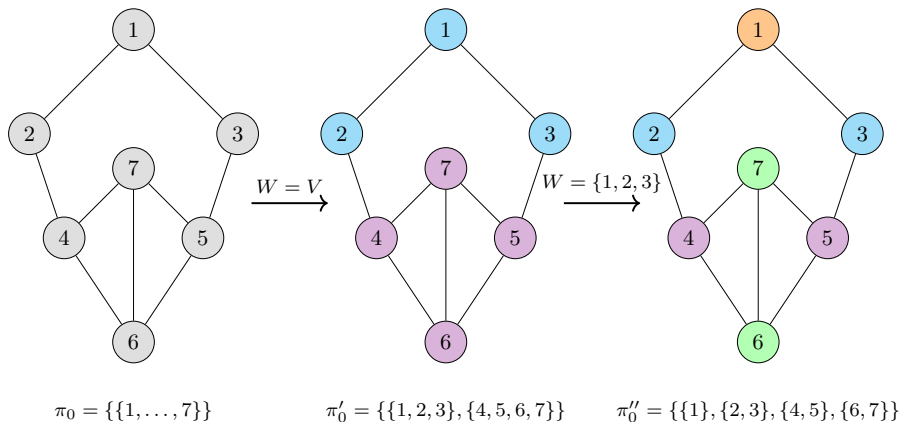
La construction se base sur 3 grandes fonctions:

- Une fonction de raffinement: Permettre de raffiner une partition (vers une partition discrète)
- Une fonction "target cell": Choisir une cellule dans la partition afin de créer les enfants de notre current node
- Une fonction d'invariant de noeud: Calculer un invariant par node afin de trouver notre forme canonique.

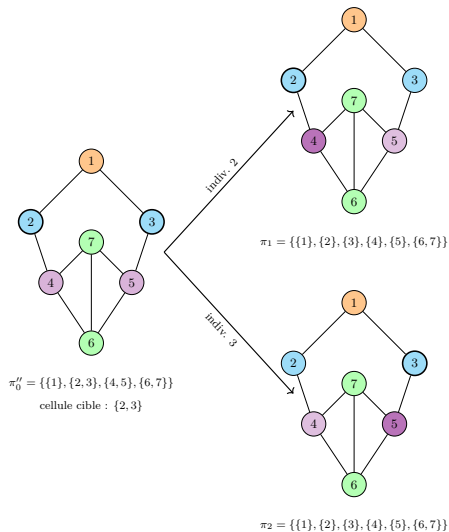
Combinaison des fonctions



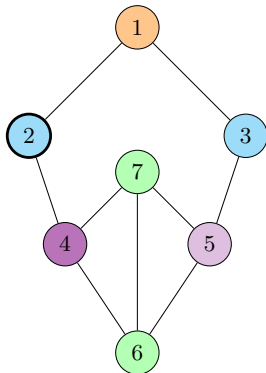
Fonction de raffinement (exemple)



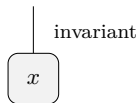
Fonction target + individualisation (exemple)



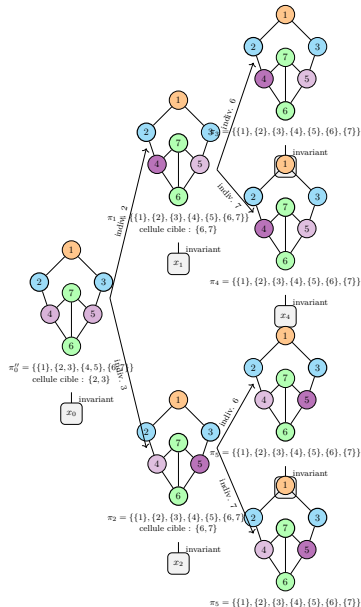
Invariant de noeud



$$\pi_3 = \{\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6, 7\}\}$$



Arbre de recherche: intermédiaire



- Plus on augmente la taille des graphes, plus l'arbre de recherche peut devenir extrêmement grand!
- Besoin de stratégies d'élagage pour retirer des noeuds inutiles
- 2 Stratégies basées sur les invariants.
- 1 stratégie basée sur le groupe d'automorphisme.

Stratégie basée sur l'invariant

$\mathcal{P}_A$

Éliminer les branches dont l'invariant ne peut pas atteindre la valeur maximale: à chaque niveau de l'arbre, on conserve le meilleur invariant trouvé jusqu'à présent, et tout autre nœud situé au même niveau dont l'invariant est strictement inférieur voit sa branche élaguée.

$\mathcal{P}_B$

Les noeuds sont comparés à un nœud de référence (noeud sur le chemin de la première feuille rencontrée). Si les deux invariants diffèrent, le sous-arbre courant est élagué. L'objectif n'est pas réellement de trouver le meilleur invariant, mais plutôt de trouver des nouveaux automorphismes.

En pratique

Il faut que les 2 conditions soient combinées pour élaguer le noeud.

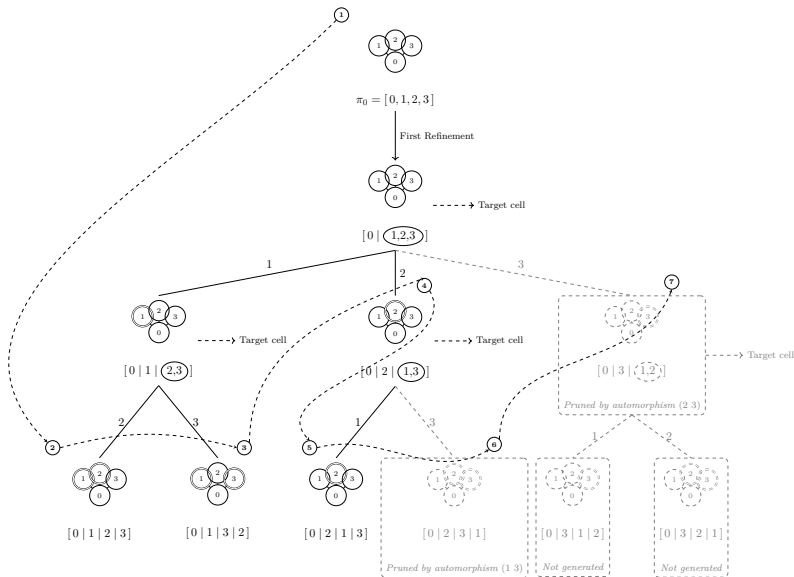
$\mathcal{P}_C$

Exploiter les automorphismes déjà découverts: si un automorphisme g associe le nœud courant σ à un nœud σ^g déjà exploré, alors les deux sous-arbres sont isomorphes. L'exploration de σ^g étant redondante, son sous-arbre est élagué.

Utilité supplémentaire

Cette stratégie permet d'avoir le groupe d'automorphisme. L'algorithme donne 2 choses: la forme canonique + groupe d'automorphisme au passage.

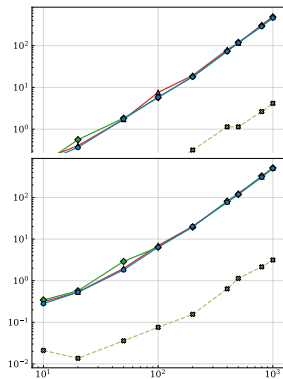
Arbre de recherche élagué



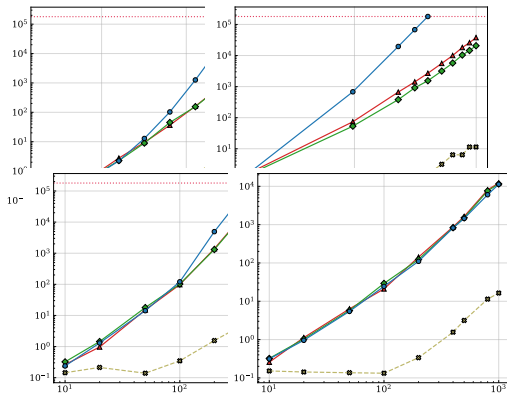
Experimentation



Seulement raffinement



Avec individualisation



- $\mathcal{P}_C$ est le plus efficace.
- $\mathcal{P}_A$ et $\mathcal{P}_B$ pas du tout performant: plusieurs raisons possible (potentielle correction)
- Conclusion: grande différence de performance avec et sans élagage.

Conclusion



- *Nauty* est un des meilleurs programmes pour résoudre le problème d'isomorphisme de graphe.
- Beaucoup plus de fonctionnalités ont été rajoutées pour optimiser le calcul de la forme canonique.
- Implémentation moins bonne que l'original mais respecte bien la logique derrière.

Merci pour votre attention!

Questions?

rodolphe.prevot@ulb.be